

System Design for Software Engineering Interviews

Lesson 1 — What Is System Design?

Core objective: Learn to reason about how complete software systems are structured, scaled, secured, monitored, and made resilient.

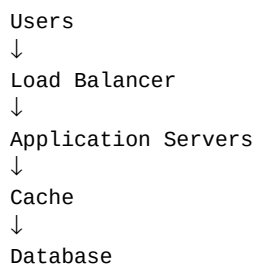
1. What Is System Design?

System design is the process of deciding how the different parts of a software system should be structured, communicate, store data, scale, and handle failures to satisfy a set of requirements.

Simple definition: System design is figuring out how to turn a software idea into an architecture that can work reliably at the required scale.

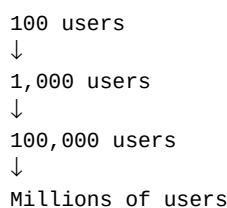
Coding asks: “How do I implement this feature?”

System design asks: “How should the entire system be structured so that this feature—and potentially millions of requests—works reliably?”



2. Why Do We Need System Design?

A simple application may initially work with one server and one database. As usage grows, the same architecture can become slow, expensive, unreliable, or impossible to scale.



Typical problems include overloaded servers, slow database queries, traffic spikes, large-file bandwidth consumption, single points of failure, and slow synchronous operations.

System design provides architectural solutions such as load balancing, caching, replication, partitioning, queues, CDNs, and redundant services.

3. System Design vs Software Development

Software development focuses on implementing components: functions, classes, modules, and services.

System design focuses on how those components interact and how the complete system behaves under load and failure.

Software development:
Function → Class → Module → Service

System design:

Users → Load Balancer → Services → Cache → Database → Queue → Workers

4. Main Goals of System Design

Goal	What it means
Scalability	The ability to handle increasing users, traffic, and data.
Reliability	The system continues to behave correctly despite problems.
Availability	The system remains operational and accessible.
Performance	The system responds within acceptable latency and throughput targets.
Maintainability	Engineers can understand, modify, test, and deploy it safely.
Security	Data and operations are protected from unauthorized access and attacks.
Fault tolerance	Failures of individual components do not unnecessarily bring down the entire system.

5. Functional vs Non-Functional Requirements

Functional requirements

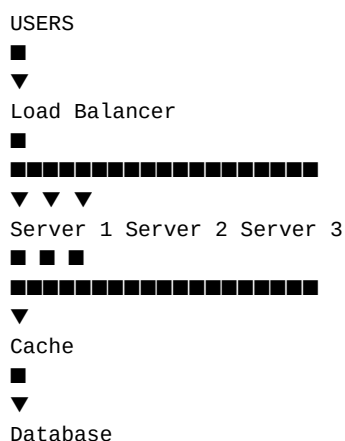
These describe **what the system does**. For a messaging application: registration, login, one-to-one messaging, group creation, message history, and delivery status.

Non-functional requirements

These describe **how the system should behave**: expected scale, latency, availability, durability, security, and capacity during traffic spikes.

Interview principle: Do not start drawing technologies immediately. First clarify what the system must do and the constraints under which it must operate.

6. High-Level Architecture



A load balancer distributes traffic. Application servers execute business logic. A cache serves frequently requested data quickly. The database provides persistent storage.

7. Think in Responsibilities, Not Technologies

A strong designer starts with the problem, not the product name. Instead of saying “I will add Redis,” ask what problem needs solving. If the database receives huge numbers of repeated reads, caching may be an appropriate solution.

Likewise, do not add Kafka simply because it is popular. First identify a need for asynchronous processing, event streaming, buffering, decoupling, or similar behavior.

Core mindset: Every component should have a clear responsibility and a reason for existing.

8. Scalability

Vertical scaling

```
8 GB RAM
↓
32 GB RAM
↓
128 GB RAM
```

Make one machine more powerful. It is simple, but hardware has limits and a single-machine failure remains a concern.

Horizontal scaling

```
Load Balancer
/ | \
↓ ↓ ↓
Server Server Server
1 2 3
```

Add more machines. This can provide substantial capacity and redundancy, but it introduces distributed-system complexity.

9. Bottlenecks

A bottleneck is a component that limits the performance of the whole system.

```
100 Application Servers
↓
Database
↓
Bottleneck
```

Adding more application servers will not solve a database bottleneck. Possible remedies include caching, indexes, read replicas, sharding, asynchronous processing, and query optimization.

Key skill: Find the bottleneck before trying to optimize the system.

10. Trade-offs — The Heart of System Design

There is rarely a universally perfect architecture. Engineering decisions involve trade-offs.

Examples include consistency vs availability, performance vs cost, and simplicity vs scalability.

A strong interview answer explains: “**Given these requirements, I choose X because... while accepting the trade-off of...**”

11. Consistency and Availability

In distributed systems, replicated data can temporarily differ because updates take time to propagate.

```
Primary
/ \
```

↓ ↓

Replica A Replica B

A → new data

B → old data (temporarily)

This introduces choices about consistency, availability, latency, and failure behavior. These ideas lead to deeper topics such as replication, consistency models, and the CAP theorem.

12. High-Level Design vs Low-Level Design

High-Level Design (HLD)

Focuses on architecture: services, databases, caches, queues, communication, scaling, reliability, and system boundaries.

Low-Level Design (LLD)

Focuses on implementation: classes, interfaces, objects, methods, OOP principles, SOLID, and design patterns.

HLD: "What should the system look like?"

LLD: "How should the components/classes be implemented?"

13. System Design Is About Evolution

Stage 1:

Client → Server → Database

Stage 2:

Client → Load Balancer → Multiple Servers → Database

Stage 3:

Client → Load Balancer → Servers → Cache → Database

Stage 4:

Client → API → Cache / Database / Queue → Workers

Architecture should evolve as requirements, traffic, data volume, and reliability expectations change. Avoid introducing complexity before it is justified.

14. The System Design Interview Mental Model

Requirements

↓

Scale

↓

API / Interface

↓

Data Model

↓

High-Level Design

↓

Identify Bottlenecks

↓

Scale It

↓

Handle Failures

↓

Security

↓

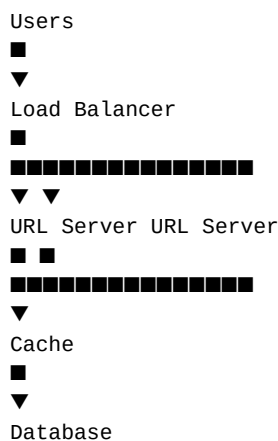
Trade-offs

15. Seven Questions to Ask

- What does the system need to do? — Functional requirements.
- How big is it? — Users, requests/sec, data volume, concurrent users.
- How fast must it be? — Latency and performance.
- What happens when something fails? — Reliability and fault tolerance.
- Where is the bottleneck? — Scalability and performance.
- What data needs to be stored? — Database and storage design.
- What trade-offs am I making? — Engineering judgment.

16. Example: URL Shortener

Suppose the user submits a long URL and receives a short identifier. Visiting the short URL should redirect to the original.



Redirects are often read-heavy, making caching valuable. Multiple application servers improve capacity and resilience. Persistent storage keeps the mapping durable.

17. Three Popular Interview Questions + Answers

Question 1 — Design a URL Shortener

Answer approach: Clarify that users create short URLs and redirect through them. Estimate scale and recognize that reads may dominate writes. Use a load balancer, multiple stateless application servers, a persistent database, and a cache for popular mappings.

Discuss unique ID generation, collision avoidance, expiration if required, database indexing, caching, high availability, and what happens if the cache or database fails.

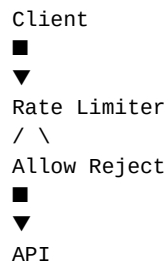
Question 2 — Design a Chat Application

Answer approach: Define one-to-one/group messaging, message persistence, offline users, delivery status, and real-time delivery. Use persistent connections such as WebSockets for live communication, a message service, durable storage, and asynchronous processing where useful.

Discuss message ordering, duplicates, retries, delivery guarantees, offline message storage, scaling concurrent connections, and database partitioning.

Question 3 — Design a Rate Limiter

Answer approach: Place the limiter before protected APIs and enforce a policy such as 100 requests/minute/user. In a multi-server environment, rate-limit state must be shared or consistently partitioned; Redis is one possible implementation.



Know fixed-window, sliding-window, token-bucket, and leaky-bucket approaches. Discuss Redis failure, clock/window issues, distributed consistency, and whether to fail open or closed.

18. Final Takeaways

System design = requirements → architecture → scale → reliability → security → trade-offs.

Do not memorize architecture diagrams as recipes. Learn to explain why each component exists, what problem it solves, what happens when it fails, and what trade-off it introduces.

Core interview mindset: “What problem am I solving, what constraint am I dealing with, and what is the simplest architecture that satisfies the requirement?”